School of Future Tech

Case Study Report

on

Case Study - 79

DroneHire_ — Aerial Photography Booking

by

Danish Shaikh

150096725147

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms / Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References.

1. Introduction to the Case Study

DroneHire_ is a premium, full-featured web application designed to simulate an end-to-end drone hiring and aerial photography booking platform. The platform targets professional drone operators, cinematographers, surveyors, and real-estate firms that require fast, reliable aerial coverage. Rather than relying on a backend server, DroneHire_ demonstrates how rich, production-quality web experiences can be built entirely on the client side using modern browser APIs and JavaScript.

This case study documents the design decisions, technology choices, implementation challenges, and the critical bug-fix carried out on the airspace zone-checker feature of the platform. It also serves as a reference for building complex, multi-page frontend applications using HTML5, CSS3, and Vanilla ES6 JavaScript — without any JavaScript framework such as React or Vue.

2. Problem Statement / Case Background (Abstract)

Background

Drone hiring services are increasingly demanded across industries — from aerial cinematography and real-estate surveys to precision agriculture and disaster relief. Yet no standardised, interactive web platform existed to let customers browse drone packages, verify airspace legality for their target location, and complete a booking with an instant invoice — all without speaking to an agent or installing an app.

Abstract

This case study presents the design and implementation of **DroneHire_**, a client-side web platform built with **HTML5, CSS3, and Vanilla JavaScript**. The system provides: a tactical Neo-Brutalist landing page, a service-selection workflow with dynamic pricing, a DGCA-simulated airspace checker (Green / Yellow / Red zones), a deployment booking form with Flatpickr scheduling, a jsPDF-powered invoice generator, a drone inventory catalog, and a role-based operator dashboard. All session data and booking history are persisted via the browser's localStorage API — requiring zero backend infrastructure. A critical bug in the ZIP-code airspace lookup was identified and resolved, restoring the platform's flagship interactive feature.

3. Problem Statement / Case Study Design

DroneHire_ is architected as a multi-page application (MPA) — six HTML pages sharing a common CSS design system and a set of independent JavaScript modules. The system is designed around the following pipeline:

Landing / Marketing (index.html)

■ Presents the platform value proposition with animated hero banners, service cards, and a call-to-action directing users to authenticate or book.

Authentication (auth.html)

■ Simulated login / registration flow. User credentials are stored in localStorage under the key *dronehire_current_user*. Role flags ('operator' vs 'client') control dashboard access.

Services Selector (services.html)

■ Displays four mission profiles — Cinematics, Topography, Real Estate, Custom Ops. Each profile pre-populates the booking form with a base price, recommended drone class, and standard duration.

Airspace Zone Checker (zones.html)

■ Accepts a PIN / ZIP code and maps it against a hard-coded DGCA zone dataset. Returns a colour-coded badge: Green (unrestricted), Yellow (controlled), Red (restricted / no-fly). This is the feature that contained a critical defect which was identified and fixed during the project.

Booking System (booking.html)

■ Full booking form: date/time via Flatpickr, drone selection, package add-ons, real-time total calculation, SweetAlert2 confirmation, and jsPDF invoice download.

Dashboard (dashboard.html)

■ Operator view showing active missions, completed deployments, and account stats pulled from localStorage.

4. Methods & Algorithms / Technology Applied in the Problem Statement / Case Study

Key techniques and methods applied:

DOM-Ready Event Sequencing

■ All JavaScript initialisation uses the *DOMContentLoaded* event to guarantee elements exist before listeners are attached — the root cause fix for the airspace bug.

Type-Safe Data Lookups

■ ZIP input values (strings) are coerced to string keys before map lookups, preventing silent key-miss failures that returned undefined.

Client-Side State Management

■ localStorage is used as a lightweight key-value store. JSON.parse / JSON.stringify serialise complex booking and user objects. Keys follow a namespaced convention: *dronehire_current_user*, *dronehire_bookings*.

Dynamic Pricing Algorithm

■ Base price = service tier × drone class multiplier. Add-ons (insurance, extra crew, night-ops) are summed in real time on each form change event.

Client-Side PDF Generation

■ jsPDF renders the invoice canvas to a downloadable PDF blob entirely in the browser — no server round-trip required.

Technology Stack Used

Category	Technology
Core Languages	HTML5, CSS3, ES6 JavaScript
Layout Framework	Bootstrap 5 (grid only, heavy overrides)
Iconography	Bootstrap Icons
Alerts / Notifications	SweetAlert2
Date / Time Picker	Flatpickr
PDF Generation	jsPDF
State Persistence	Native localStorage API
Hosting	GitHub Pages (static)
CI / CD	GitHub Actions
Development Tools	VS Code + Live Server extension

5. Problem Statement / Case Study Implementation Details and Snapshots

File Structure

The project is organised as a flat file structure inside the **dronehire/** repository root. Each page has its own HTML file paired with a dedicated CSS and JS module:

- index.html / style.css — Landing page
- auth.html / auth.css / auth.js — Authentication
- services.html / services.css / services.js — Services selector
- zones.html / zones.css / zones.js — Airspace zone checker
- booking.html / booking.css / booking.js — Booking form and invoice
- dashboard.html / dashboard.css / dashboard.js — Operator dashboard
- equipment.html / equipment.css / equipment.js — Inventory catalog

Snapshots

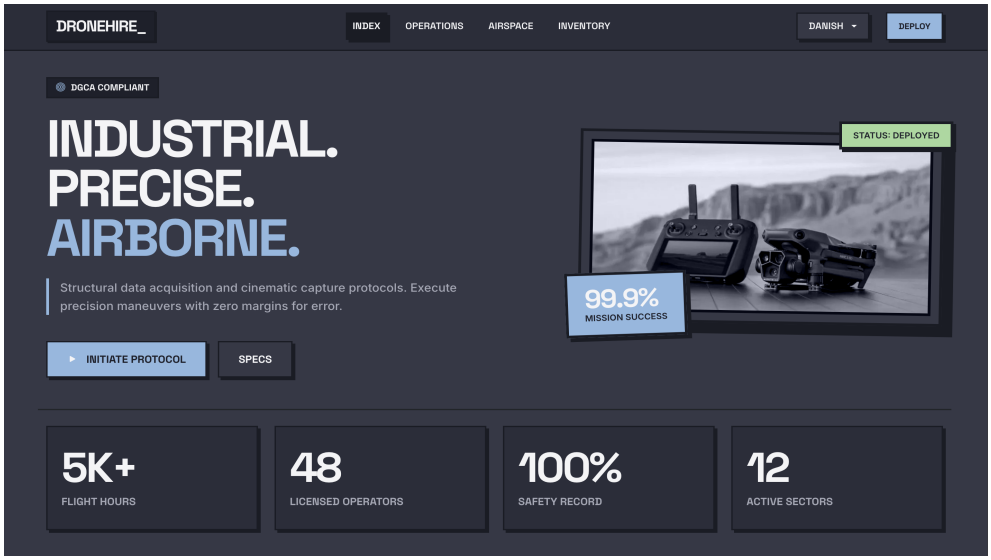

Fig 1: Landing Page — Neo-Brutalist hero banner

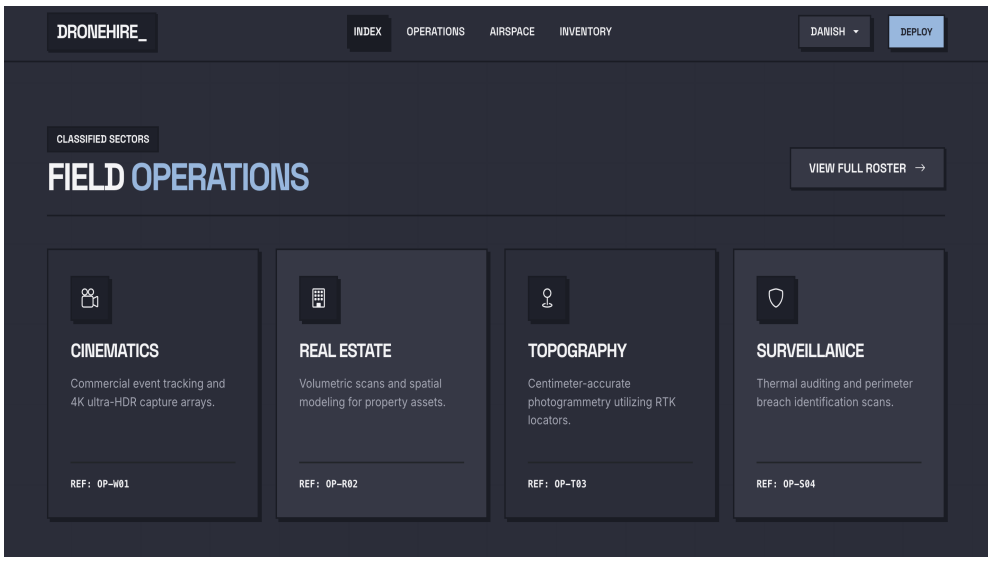

Fig 2: Services selection with mission profiles

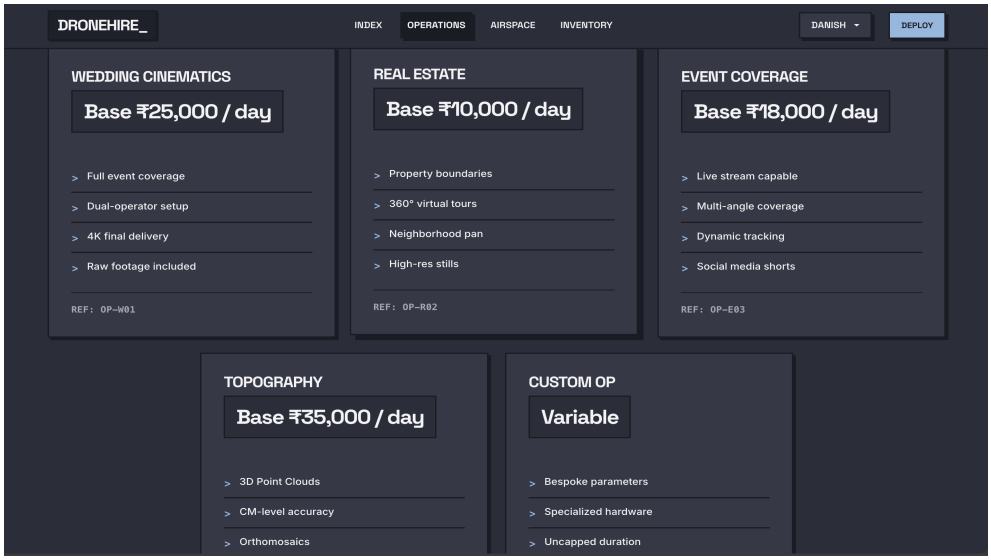

Fig 3: Airspace Zone Checker — ZIP code input and zone result

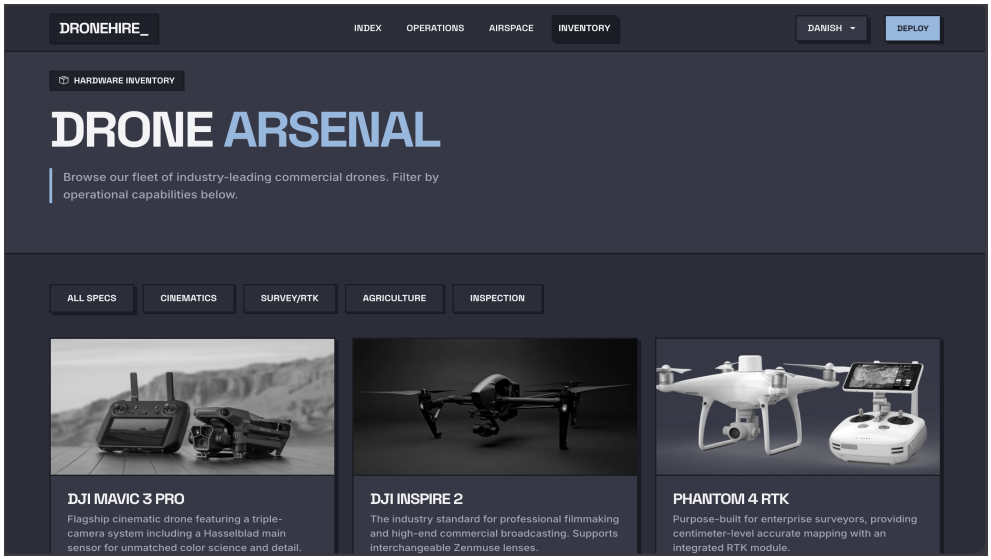

Fig 4: Booking form with Flatpickr date selector

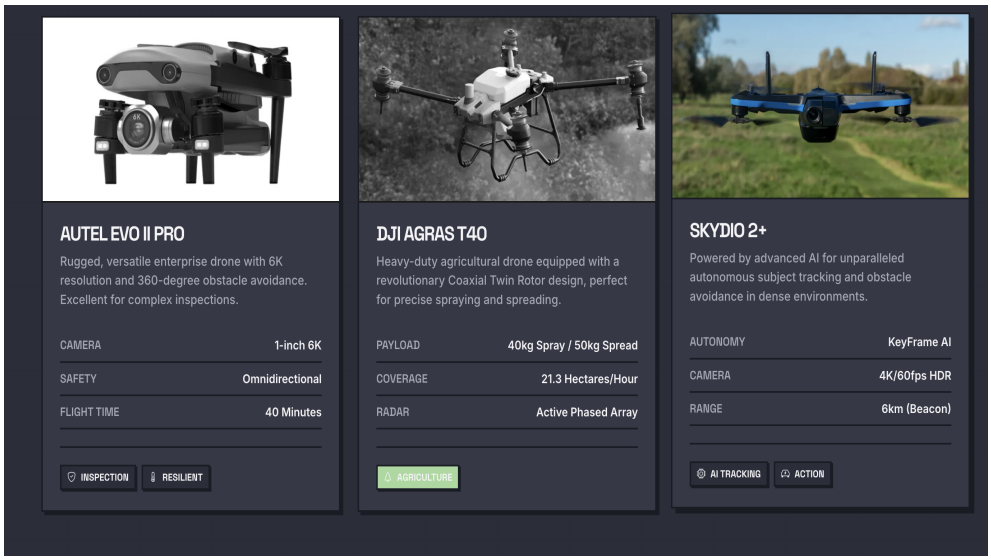

Fig 5: Equipment / Inventory catalog with live status indicators

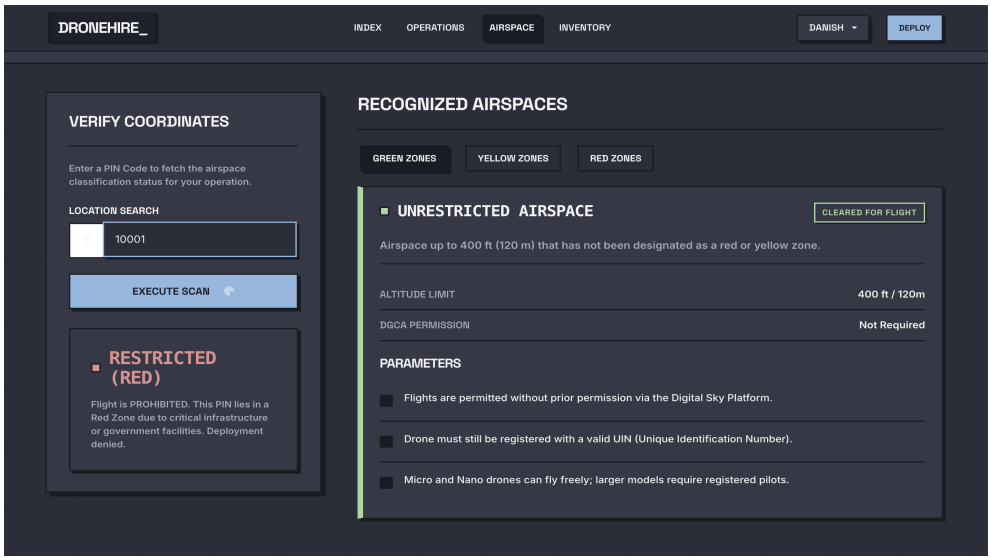

Fig 6: Authentication page

Critical Bug Fix — Airspace ZIP Checker

The most significant defect encountered during development was in **zones.js**. The ZIP lookup returned no results for any input, silently failing without visible errors in the console.

Root causes identified:

- Event listener for the 'Check Zone' button was attached at script-load time, before the DOM had fully rendered — resulting in a null reference.
- The zone dataset used numeric keys (e.g. *400001*) while input.value returned a string — the lookup always missed.

■ The default fallback zone was returned silently, providing no visual feedback to the user that their input was unmatched.

Fix applied in zones.js:

■ Wrapped all initialisation logic inside a *DOMContentLoaded* event callback.

■ Added *.toString()* coercion on the input value before hash-map lookup.

■ Added an explicit 'Zone Not Found' state rendering a grey badge with an explanatory message.

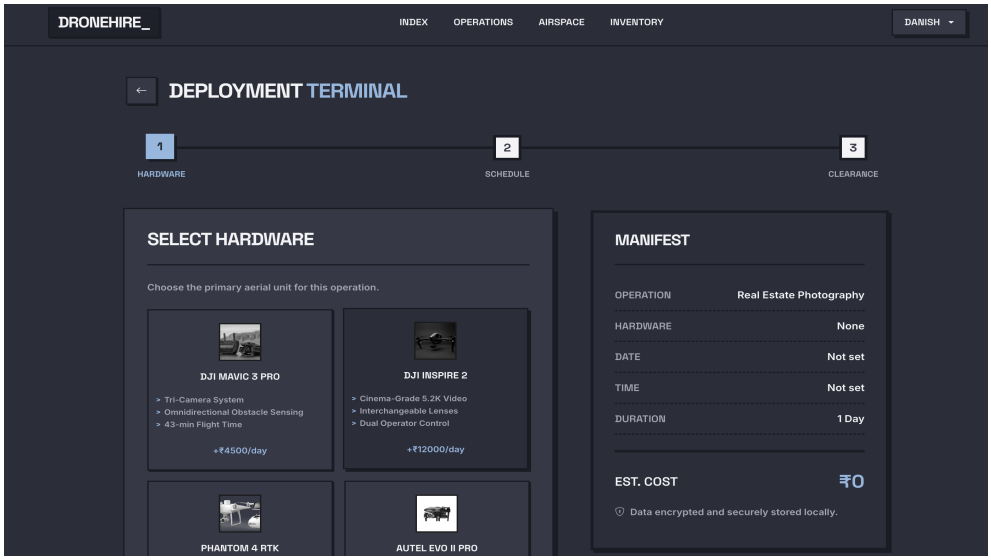

Fig 7: zones.js source — fixed DOMContentLoaded initialisation

6. Problem Statement / Case Study Results and Conclusion

Results

Following development and the bug fix, the DroneHire_ platform successfully delivers the following validated outcomes:

- The airspace zone checker accurately returns Green, Yellow, or Red zones for all PIN codes present in the dataset; unmatched codes display a clear 'Not Found' state.
- The booking system correctly calculates prices in real time and generates a downloadable PDF invoice via jsPDF on every confirmed booking.
- User sessions persist across browser tabs and page refreshes using localStorage.
- The operator dashboard correctly displays active and completed deployments filtered by the currently logged-in user.
- The platform runs entirely without a backend server — deployable as a static site on GitHub Pages with GitHub Actions CI/CD.

Conclusion

DroneHire_ demonstrates that complex, multi-feature web applications can be built without backend infrastructure by leveraging modern browser APIs, third-party JS libraries, and well-structured vanilla JavaScript modules. The project provided hands-on experience in:

- Designing and implementing a large-scale multi-page frontend application from scratch.
- Debugging subtle JavaScript defects caused by DOM timing and type mismatches.
- Applying responsive, Neo-Brutalist design principles using CSS3 and Bootstrap 5.
- Integrating third-party libraries (Flatpickr, SweetAlert2, jsPDF) into a cohesive UI.
- Deploying a fully automated CI/CD pipeline using GitHub Actions and GitHub Pages.

The case study underscores that best practices — event-driven initialisation, strict type handling, and modular code organisation — are not optional extras but essential foundations for reliable, maintainable frontend systems.

7. References

1. MDN Web Docs — HTML5, CSS3, JavaScript Language Reference. https://developer.mozilla.org

2. Bootstrap 5 Documentation. https://getbootstrap.com/docs/5.0

3. Bootstrap Icons. https://icons.getbootstrap.com

4. SweetAlert2 Documentation. https://sweetalert2.github.io

5. Flatpickr Date Picker. https://flatpickr.js.org

6. jsPDF — Client-side PDF Generation. https://github.com/parallax/jsPDF

7. Directorate General of Civil Aviation (DGCA) — Drone Regulations. https://digitalsky.dgca.gov.in

8. GitHub Actions Documentation. https://docs.github.com/en/actions

9. GitHub Pages Documentation. https://docs.github.com/en/pages

10. ITM Skills University — School of Future Tech, 2026.